

Building a Background PDF Generation App Using Python Celery and Template Data

Last Updated : 23 Jul, 2025



In today's digital age, businesses often require the generation of PDF files with dynamic data for various purposes such as reports, invoices, or certificates. However, generating these PDF files synchronously within a web application can lead to performance issues and user experience degradation. To solve this, we can create a background app that asynchronously generates PDF files with data from templates. Here, we will build such an app using Celery and [Flask](#), two popular [Python](#) frameworks.

Step-by-Step Implementation

Project Structure

```
pdf_generator/  
├── app.py  
├── celery_config.py  
├── tasks.py  
└── templates/  
    └── index.html
```

Step 1: Setting Up the Environment

```
pip install Flask Celery redis Jinja2 reportlab
```

Step 2: Defining Celery Configuration

In celery_config.py, define the Celery configuration, including the broker and backend URLs.

```
from celery import Celery  
  
celery_app = Celery(  
    'tasks',  
    broker='redis://localhost:6379/0',  
    backend='redis://localhost:6379/0',  
)
```

Step 3: Defining Celery Tasks

In tasks.py, define the Celery task to generate PDF files. We'll use the ReportLab library to create PDFs with dynamic data.

```
from celery_config import celery_app  
from reportlab.lib.pagesizes import letter  
from reportlab.pdfgen import canvas  
  
@celery_app.task  
def generate_pdf():  
    # Generate a PDF file  
    filename = 'example.pdf'  
    c = canvas.Canvas(filename, pagesize=letter)  
    c.drawString(100, 750, "Hello, this is a PDF generated  
by Celery!")  
    c.save()  
    return filename
```

Step 3: Creating the Flask Application

In app.py, create a Flask application and configure it to use Celery. Define routes to trigger PDF generation tasks.

```
from flask import Flask, render_template  
from tasks import generate_pdf  
  
app = Flask(__name__)  
  
# Load Celery configuration  
app.config.update(  
    CELERY_BROKER_URL='redis://localhost:6379/0',  
    CELERY_RESULT_BACKEND='redis://localhost:6379/0'  
)  
  
# Initialize Celery  
from celery import Celery  
celery = Celery(app.name,  
    broker=app.config['CELERY_BROKER_URL'])  
celery.conf.update(app.config)  
  
@app.route('/')  
def index():  
    return render_template('index.html')  
  
@app.route('/generate-pdf')  
def trigger_generate_pdf():  
    result = generate_pdf.delay()  
    return f'PDF generation task queued! Task ID:  
{result.id}'  
  
if __name__ == '__main__':  
    app.run(debug=True)
```

Step 4: Implementing the PDF Generation Logic

Create a template HTML file (index.html) in the templates directory. This file will contain the structure of the PDF file. Use placeholders for dynamic data.

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>PDF Generator</title>  
</head>  
<body>  
    <h1>Welcome to the PDF Generator!</h1>  
    <p>Click the button below to generate a PDF file.</p>  
    <form action="/generate-pdf" method="get">  
        <button type="submit">Generate PDF</button>  
    </form>  
</body>  
</html>
```

Step 5: Run the application

- Start a Redis server.
- Run the Celery worker by executing celery -A celery_config.celery_app worker --loglevel=info.
- Run the Flask application by executing python app.py.

Access the Flask application in browser at <http://127.0.0.1:5000> and click the "Generate PDF" button to trigger the Celery task to generate a PDF file.

Interpret the Result

- "PDF generation task queued!": This part of the output confirms that the task to generate a PDF file has been queued for execution.
- "Task ID: Each task in Celery is assigned a unique identifier (UUID) known as the task ID. This ID is used to track the progress and status of the task. In this case, "dc86e58c-abcb-47fe-8f83-10f0590a9773" is the task ID assigned to the PDF generation task.
- Overall, this output indicates that the PDF generation process has been initiated and is awaiting execution by the Celery worker. Once the task is picked up by the worker, it will generate the PDF file asynchronously in the background.

 Comment

 pmishr...

 0



Article Tags: [Python](#) [Python Flask](#)

Explore

Python Fundamentals	▼
Python Data Structures	▼
Advanced Python	▼
Data Science with Python	▼
Web Development with Python	▼
Python Practice	▼



 **Corporate & Communications Address:**
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

 **Registered Address:**
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Explore

POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data
Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design